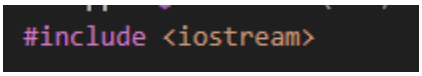
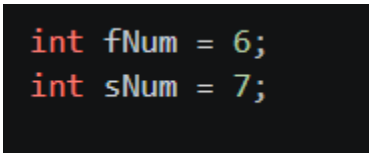
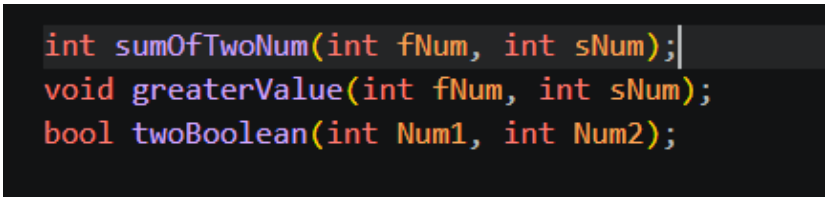
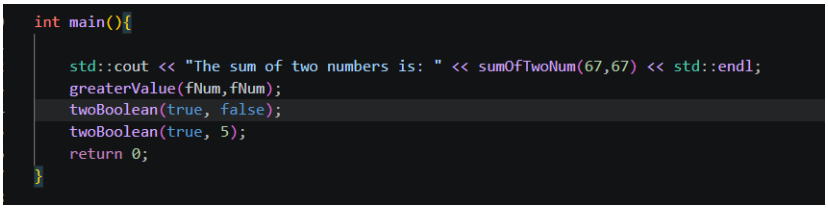
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/9/2026	**DATE SUBMITTED:**	7/13/2026
NAME:	Gabriel Immanuel R. Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

Sections	Answer
Header File Declaration Section	
Global Declaration Section	
Class Declaration and Method Definition Section	
Main Function	

```
#include <iostream>
```

```
int fNum = 6;
int sNum = 7;
```

```
int sumOfTwoNum(int fNum, int sNum);
void greaterValue(int fNum, int sNum);
bool twoBoolean(int Num1, int Num2);
```

```
int main(){

    std::cout << "The sum of two numbers is: " << sumOfTwoNum(67,67) << std::endl;
    greaterValue(fNum,fNum);
    twoBoolean(true, false);
    twoBoolean(true, 5);
    return 0;
}
```

Method Definition	

```cpp
int sumOfTwoNum(int fNum, int sNum){
    int sum = fNum + sNum;
    return sum;
}


void greaterValue(int fNum, int sNum){
    if (fNum > sNum){
        std::cout << "The "<< fNum << " is greater than the " << sNum << std::endl;
    } else{
        std::cout << "The "<< sNum <<" is greater than the " << fNum << std::endl;
    }
}

bool twoBoolean(int Num1, int Num2){

    if (Num1 != 0 && Num1 != 1){
        std::cout << "The checking was unsuccessful, please input a valid boolean value.\n";
        return false;
    }
    else if (Num2 != 0 && Num2 != 1){
        std::cout << "The checking was unsuccessful, please input a valid boolean value.\n";
        return false;
    }

    std::cout << "\nTrue(1) and False(0)" << std::endl;
    std::cout << "True or False: " << (Num1 && Num2) << std::endl;
    std::cout << "True or False: " << (Num1 || Num2) << std::endl;
    std::cout << "True or False: " << (!Num1) << std::endl;
    std::cout << "True or False: " << (!Num2) << std::endl;


    std::cout << "The checking was successful" << std::endl;

    return true;
}
```

The code uses function prototypes and function definitions to complete different tasks. Each function prototype has its own parameters and purpose. It contains functions to calculate the sum of two numbers, determine which of the two numbers is greater, and perform logical operations on two Boolean values. The main function purpose is to call each of these functions and display their outputs based on the values passed as arguments or parameters. For example, I passed the values 67 and 67 to the sumOfTwoNum function, so it adds the two numbers and displays their sum. The same goes for the greaterValue function, it checks whether the value is greater than or less than by using an if statement. Lastly, the twoBoolean function checks whether the given arguments are valid Boolean values and if the conditions aren't met it will display an error message.

Table 1-1. C++ Structure Code for Answer

```cpp
#include <iostream>

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
        Triangle(double A, double B, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
};
```

The code creates a Triangle class that contains both private and public members. The private variables store all the angles of the triangle, and it cannot be accessed from outside the class. While the public variables contain all the functions that allow the user to create a triangle, set an angle for the triangle, and check if it's a valid triangle.

```cpp
int main(){
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
    }   else{
            std::cout << "The shape is NOT a valid triangle.\n";
    }


    return 0;
}
```

The main function is where the program creates a Triangle object by passing values to the constructor. It then calls the validateTriangle function to check whether the given angles form a valid triangle or not. This is where you usually use all the classes that you defined.

```cpp
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}


void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}


const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

This is the method definition section, where the constructor and the function of the Triangle are defined. The constructor initializes three angles and calculates their total angle, while the setAngles method lets you update the angle value and recalculates the total angle of the triangle. Lastly, the validateTriangle function checks whether the total angle is less than or equal to 180.

Table 1-2. ILO B output observations and comments.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. **Create a C++ program to swap the two numbers in different variables.**

```cpp
#include <iostream>

int main(){

    int a = 67;
    int b = 69;
    int temp = 0;

    std::cout << "Before swapping values" << std::endl;
    std::cout << "The value of a is: " << a << std::endl;
    std::cout << "The value of b is: " << b << std::endl;

    temp = a;
    a = b;
    b = temp;

    std::cout << "\nAfter swapping values" << std::endl;
    std::cout << "The value of a is: " << a << std::endl;
    std::cout << "The value of b is: " << b << std::endl;

    return 0;
}
```

Output:

```
Before swapping values
The value of a is: 67
The value of b is: 69

After swapping values
The value of a is: 69
The value of b is: 67
PS C:\Users\Gabriel\Desktop\CPP>
```

Code analysis :

The code starts by declaring two integer variables along with a temporary variable that is used to swap their values. It first displays the original values, then assigns the first value to the temporary variable before exchanging the values of the two variables. Lastly, it prints the new values to show that the numbers have been successfully swapped.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
#include <iostream>

double kelvinToFahrenheit(double kelvin);

int main(){

    double kelvin = 300;

    std::cout << "The Kelvin temperature is: " << kelvin << std::endl;
    std::cout << "The temperature in Fahrenheit is: " << kelvinToFahrenheit(kelvin) << std::endl;

    return 0;
}

double kelvinToFahrenheit(double kelvin){
    double fahrenheit = (kelvin - 273.15) * 9/5 + 32;
    return fahrenheit;
}
```

Output

```
The Kelvin temperature is: 300
The temperature in Fahrenheit is: 80.33
PS C:\Users\Gabriel\Desktop\CPP>
```

Code analysis:

The code uses a function prototype and a function definition to convert a temperature from Kelvin to Fahrenheit. It initializes the Kelvin value to 300 in the main function, then passes it to the kelvinToFahrenheit function, which performs the conversion and returns the Fahrenheit value. Lastly, the program displays both the original Kelvin temperature and the converted Fahrenheit temperature.

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>

double distanceTwoPoints(double x1, double y1, double x2, double y2);

int main(){
    double x1, y1, x2, y2;

    std::cout << "Enter the first distance(x): ";
    std::cin >> x1;
    std::cout << "Enter the first distance(y): ";
    std::cin >> y1;

    std::cout << "\nEnter the second distance(x): ";
    std::cin >> x2;
    std::cout << "Enter the second distance(y): ";
    std::cin >> y2;

    std::cout << "\nThe distance between the two points is: " << distanceTwoPoints(x1, y1, x2, y2) << std::endl;

    return 0;
}

double distanceTwoPoints(double x1, double y1, double x2, double y2){
    double distance = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    return distance;
}
```

Output

```
Enter the first distance(x): 10
Enter the first distance(y): 20

Enter the second distance(x): 30
Enter the second distance(y): 40

The distance between the two points is: 28.2843
PS C:\Users\Gabriel\Desktop\CPP>
```

Code analysis:

The code uses a function prototype and a function definition to calculate the distance between two points. It first asks the user to enter the coordinates of two points, then passes the values to the distanceTwoPoints function. Inside the function, the distance formula is calculated using the sqrt and multiplying both the (x2-x1) and (y2-y1) coordinates to themselves to get the squared. These values are then added together to get the computed distance, which is then returned and displayed in the main.

4. Modify the code given in ILO B and add the following functions:

ILO4.cpp > ...

```cpp
#include <iostream>
#include <cmath>

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC, sideA, sideB, sideC;
    public:
        Triangle(double A, double B, double C, double side1, double side2, double side3);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
        double computePerimeter();
        double computeArea();
        void triangleInfo();

};


int main(){
    Triangle set1(40, 30, 110, 5, 5, 5);
    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
    }   else{
            std::cout << "The shape is NOT a valid triangle.\n";
    }

    std::cout << "The perimeter is: " << set1.computePerimeter() << std::endl;
    std::cout << "The area is: " << set1.computeArea() << std::endl;
    set1.triangleInfo();

    return 0;
}
```

```cpp
Triangle::Triangle(double A, double B, double C, double side1, double side2, double side3){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;

    sideA = side1;
    sideB = side2;
    sideC = side3;
}


void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
```

```cpp
const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}

double Triangle::computePerimeter(){
    return sideA + sideB + sideC;
}

double Triangle::computeArea(){
    double sP = (sideA + sideB + sideC) / 2;
    return sqrt(sP * (sP - sideA) * (sP - sideB) * (sP - sideC));
}

void Triangle::triangleInfo(){
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout << "This is an acute triangle.\n";
    }
    else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout << "This is an obtuse triangle.\n";
    }
    else{
        std::cout << "Others\n";
    }
}
```

Output :

```
The shape is a valid triangle.
The perimeter is: 15
The area is: 10.8253
This is an obtuse triangle.
PS C:\Users\Gabriel\Desktop\CPP>
```

Code analysis:

The code creates a Triangle class that stores the angles and side lengths of a triangle. It also includes functions that allow the user to create a triangle, set its angles, check whether it is valid, compute its perimeter and area, and determine its type. The constructor initializes all the triangle's angles and side lengths, this is where the values are passed and stored. The validateTriangle function checks whether the given angle from the constructor is within the range of a valid triangle. The computePerimeter function adds all the sides, while the computeArea function divides the perimeter by 2 and uses the Heron formula to get the area since we only have the lengths of the three sides. Lastly, the triangleInfo function checks whether the values on the class definition fit within the acute, obtuse, or others by using an if-else statement with logical operators.

Conclusion :

I learned how to apply the basic concepts of C++ programming by using functions, function prototypes, classes, and constructors. Throughout the procedure, all of the ILOs helped me recall how function prototypes work and how to properly define them. I had trouble with ILO A because I declared my boolean function as void, which caused it to return nothing and prevented my program from working. ILO B helped me better understand how classes and constructors work because I was able to apply it to ILO C. The tasks in ILO C were quite difficult, even though they only required the basic implementation of the concepts I had previously learned in C++. It became a major roadblock because I had forgotten how to apply those concepts to solve different programming problems. Overall, this activity was a great refresher on the fundamentals of C++. I also believe that I did okay on this activity, but I still need to improve my skills in programming tasks more.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT and Grammarly was used to help with the Grammar correction on the conclusion and analysis**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.